ShanghaiTech University

CS240 Algorithm Design and Analysis

Fall 2025

Problem Set 2

Due: 23:59, Oct. 31, 2025

1. Submit your solutions to the course Gradescope.
2. If you want to submit a handwritten version, scan it clearly.
3. Late homeworks submitted within 24 hours of the due date will be marked down 25%. Homeworks submitted more than 24 hours after the due date will not be accepted unless there is a valid reason, such as a medical or family emergency.
4. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

In the intelligent text editing system m-Pad, the document consists of lowercase characters '`m`'.

Initially, the document contains only one '`m`'. The m-Pad system is extremely minimalistic and supports only two commands:

- **Copy All**: Copies the entire content of the current document into the clipboard. (Partial copy is not allowed; the system always copies the full text.)
- **Paste**: Appends the clipboard content to the end of the document.

Your task is to determine the minimum number of operations required to make the document contain exactly n characters '`m`' by combining these two commands in an optimal way. Constraints: $1 \leq n \leq 1000$

Please solve this problem using Dynamic Programming.

Example 1

Input: `n = 1`
Output: `0`

Example 2

Input: `n = 9`
Output: `6`
Explanation:

1. Copy All → clipboard: `m`
2. Paste → document: `mm`
3. Paste → document: `mmm`
4. Copy All → clipboard: `mmm`
5. Paste → document: `mmmmmm`
6. Paste → document: `mmmmmmmmm`

Solution:

State Definition

Define $OPT[i]$ as the minimum number of operations required to generate exactly i characters '`m`'.

State Transition Equation

- Boundary conditions: When $i = 1$, there is already one '`m`' initially, so $OPT[1] = 0$.
- Case 1: If we only use "Copy All" the first time and perform "Paste" operations for the rest, we need $1 + (i - 1) = i + 1$ operations.Thus, initialize $OPT[i] = i$.
- Case 2:

  If i is divisible by some j (i.e., $i = j \times k$ for $k \geq 2$), it means we can generate j '`m`'s first (which takes $OPT[j]$ operations).

  $\rightarrow$ Perform 1 "Copy All" operation to copy the j '`m`'s to the clipboard.

  $\rightarrow$ Perform $k-1$ "Paste" operations (to append j '`m`'s $k-1$ times, resulting in $j \times k = i$ 'm's).

  Thus, the total operations for this path is $OPT[j]+1$ (Copy All) + $(k-1)$ (Paste) = $OPT[j] + k$. Since $\frac{i}{j}$, this can also be written as $OPT[j] + \frac{i}{j}$. Moreover, when multiple sets of factors exist for a given i, select the set with the minimum number of selections.

Thus, State transition equation is

$$OPT(i) = \begin{cases} 0 & i = 1 \\ \min\{i, \min_{\substack{1 \leq j < i \\ i\%j=0}}\{OPT(j) + \frac{i}{j}\}\} & i \geq 2 \end{cases}$$

Thus, the minimum number of operations required to make the document contain exactly n characters '`m`' by combining these two commands in an optimal way is $OPT(n)$.

Problem 2:

Xiaoxiong encounters n monsters sequentially, numbered from 1 to n. Monster i has health a_i and requires p_i stamina to defeat. Xiaoxiong starts with an initial stamina of P.

For each monster, Xiaoxiong has two choices:

1. **Let the monster go** – gain i experience points, and **no stamina is consumed**.
2. **Defeat the monster** – consumes p_i stamina, and gains

$$\text{experience} = a_i + (x \bmod 10) \cdot a_i$$

   where x is the total number of monsters defeated so far (including the current monster).

Note: If Xiaoxiong's remaining stamina is less than p_i, she **cannot defeat the monster** and can only let it go.

Goal: Determine the maximum total experience Xiaoxiong can earn.

Solution:

State Definition Define $OPT(i, p, k)$ as the maximum experience gained when the remaining stamina is p after dealing with the first i monsters, and the number of defeated monsters modulo 10 is k.

Here, $i \in [0, n], p \in [0, P], k \in [0, 9]$

State Transition Equation

- Boundary conditions: Initially encountered no monsters. Defeated monsters modulo $10 = 0$. Experience is 0. $OPT(0, p, k) = 0$. Other states are unreachable.$OPT(0, p, k) = -\infty, i = 0$ but $k \neq 0$

- Case 1:

  When defeating monster i requires stamina p_i exceeding Xiaoxiong's remaining stamina ($p_i > p$), monster i must be let go, and $OPT(i, p, k) = OPT(i - 1, p, k) + i$.

- Case 2:

  Defeating monster i requires stamina $p_i \leq$ Xiaoxiong's remaining stamina. You may choose whether to let monster i go:

  - let monster i go: $OPT(i, p, k) = OPT(i - 1, p, k) + i$
  - defeat the monster: Xiaoxiong can get $a_i + a_i * k$ experience, $OPT(i, p, k) = OPT(i - 1, p - p_i, (k - 1) \mod 10) + a_i + a_i * k$

Thus, State transition equation is

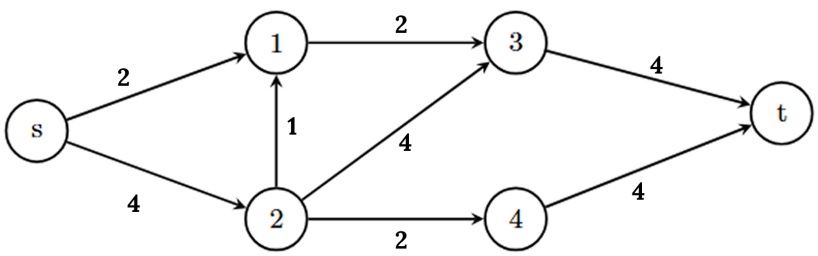
$$OPT(i, p, k) =$$

$$\begin{cases} 0 & i = 0, k = 0 \\ -\infty & i = 0 \text{ but } k \neq 0 \\ OPT(i-1, p, k) + i & p_i > p \\ \max\{OPT(i-1, p, k) + i, OPT(i-1, p - p_i, (k-1) \mod 10) + a_i + a_i * k\} & p_i \leq p \end{cases}$$

Thus, the maximum total experience Xiaoxiong can earn is $\max_{k \in [0,9]} OPT(n, P, k)$.

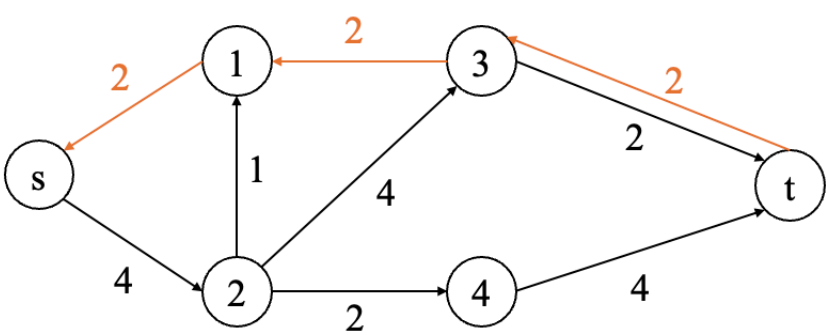
Problem 3:

Execute the Ford-Fulkerson algorithm on the flow network shown below. Display the residual network after each flow update and determine the maximum flow at the end. During each iteration, choose the lexicographically smallest available augmenting path. For example, if two augmenting paths are $s \to 2 \to t$ and $s \to 3 \to t$, select $s \to 2 \to t$ since it is lexicographically smaller. Likewise, if the paths $s \to 3 \to t$ and $s \to 2 \to 4 \to t$ are available, choose $s \to 2 \to 4 \to t$. Moreover, if all compared vertices coincide up to the end of the shorter path, the shorter prefix path is considered smaller. For example, between $s \to 1 \to t$ and $s \to 1 \to 4 \to t$, select $s \to 1 \to t$.

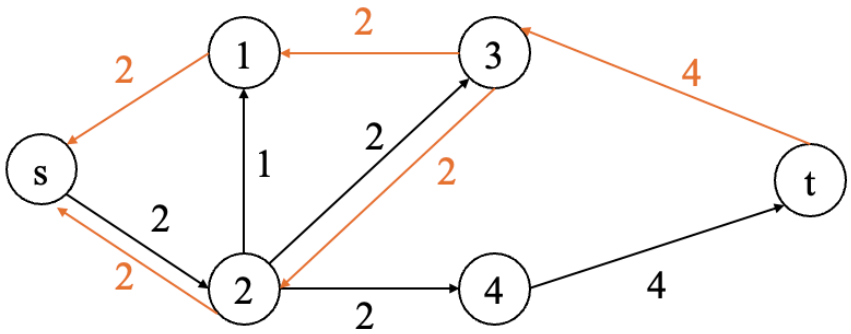


Solution:

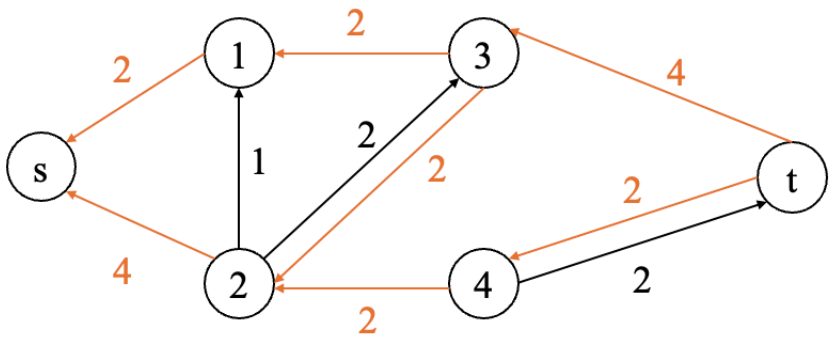
Step1: the augmenting path is s $\to$ 1 $\to$ 3 $\to$ t. The capacity of this path is min(2,2,4)=2.



Step2: the augmenting path is s $\to$ 2 $\to$ 3 $\to$ t. The capacity of this path is min(4,4,2)=2.



Step3: the augmenting path is s → 2 → 4 → t. The capacity of this path is min(2,2,4)=2.



The final maximum flow = 6

Problem 4:

You are a commander about to participate in an simulated combat exercise. You have n elite soldiers under your command, numbered from 1 to n.

There are two main battle zones on the exercise map: Zone A and Zone B. The battlefield conditions differ between these two zones, so the same soldier may have different combat effectiveness depending on where they are deployed:

- If the i-th soldier is deployed to Zone A, he contributes a_i combat power.
- If deployed to Zone B, he contributes b_i combat power.

Each soldier can only be assigned to one battle zone.

There are m synergistic groups, denoted as $g_1, g_2, \ldots, g_m$, where each group contains several specific soldiers. If all members of group g_i are deployed to the same zone, the following synergy bonuses apply:

- If all are deployed to Zone A, an additional $c_{i,1}$ combat power is gained.
- If all are deployed to Zone B, an additional $c_{i,2}$ combat power is gained.

Your task is to deploy all soldiers to Zones A and B in such a way that the total combat power (the sum of the contributions of all soldiers plus the synergy bonuses) is maximized.

For example,

$$n = 3, \quad a = [4, 2, 1], \quad b = [2, 3, 3],$$

$$m = 1, \quad g_1 = \{1, 2\}, \quad c_1 = [4, 2].$$

If soldiers 1 and 2 are deployed to Zone A and soldier 3 is deployed to Zone B, the maximum total combat power is

$$4 + 2 + 4 + 3 = 13.$$

Please use **network flow** to solve this problem. **Your answer should include the following:**

1. The construction of the nodes in the graph.
2. The way edges are connected and how capacities are defined.
3. How to calculate the maximum total combat power.
4. A diagram showing the graph constructed for the example data (similar to the illustration in *Lecture 9, page 16*).

Solution:

1 The construction of the nodes in the graph.

- Source s represents Zone A.
- Sink t represents Zone B.
- For each soldier $i = 1 \dots n$, a node u_i.
- For each group $g_k = 1 \dots m$, two auxiliary nodes:
  - p_k: "all in A" event node.
  - q_k: "all in B" event node.

2 The way edges are connected and how capacities are defined.

Soldier choice edges

- $s \to u_i$, capacity a_i: cut if soldier i not in A (lose a_i).
- $u_i \to t$, capacity b_i: cut if soldier i not in B (lose b_i).

Synergy bonus edges

- $s \to p_k$, capacity $c_{k,1}$: cut if not all in A (lose $c_{k,1}$).
- $q_k \to t$, capacity $c_{k,2}$: cut if not all in B (lose $c_{k,2}$).

Enforcement edges (infinite capacity)

- $p_k \to u_i$ for all $i \in g_k$, capacity ∞: ensures p_k in s-side only if all u_i in s-side.
- $u_i \to q_k$ for all $i \in g_k$, capacity ∞: ensures q_k in t-side only if all u_i in t-side.

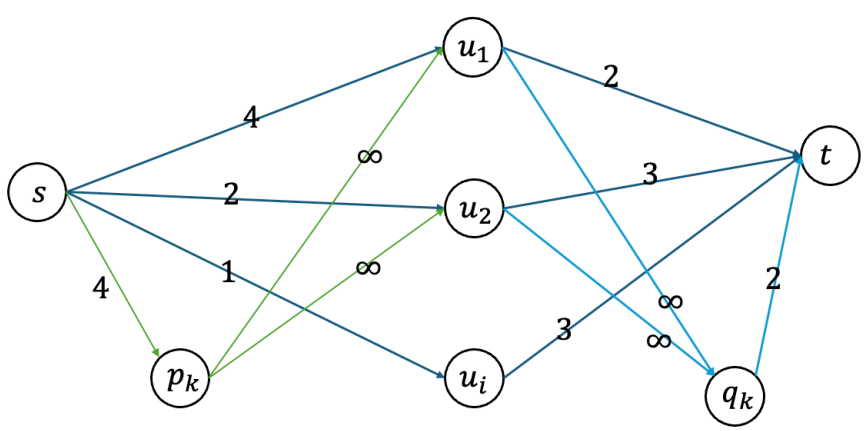
3 How to calculate the maximum total combat power

The core idea of this problem is to transform the maximization of combat power into a minimization of lost potential via a minimum cut model. We first calculate the theoretical maximum sum if all individual contributions and all synergy bonuses could be obtained simultaneously. Since deployment constraints make this impossible, we build a flow network where the capacities of the edges represent the penalties for not achieving certain contributions. The minimum cut of this network corresponds to the minimum total loss we must incur. Therefore, subtracting this min-cut value from the theoretical maximum yields the actual maximum achievable combat power.

We use the **min-cut** model in a flow network:

1. **Construct the graph** as described, with:
   - Source s and sink t
   - Soldier nodes u_i
   - Group event nodes p_k (all-in-A) and q_k (all-in-B)
   - Edges with capacities $a_i, b_i, c_{k,1}, c_{k,2}, \infty$ as defined
2. **Compute the maximum flow** from s to t (using any max-flow algorithm, e.g., Edmonds–Karp or Dinic)
3. **Define the total "potential" sum**:
$$\text{Sum} = \sum_{i=1}^{n}(a_i + b_i) + \sum_{k=1}^{m}(c_{k,1} + c_{k,2})$$
   This is the total combat power if we could get all individual contributions and all synergy bonuses simultaneously (which is impossible due to deployment constraints)
4. **Then:**
$$\text{Max Combat Power} = \text{Sum} - \text{Max Flow Value}$$
   Here, the **max flow value** equals the **min-cut capacity**, which represents the minimum total "loss" of combat power we must suffer due to conflicting deployment choices

4 A diagram showing the graph constructed for the example data



Max Combat Power $= \sum_{i=1}^{n}(a_i + b_i) + \sum_{k=1}^{m}(c_{k,1} + c_{k,2}) - \text{Max Flow Value}$ $= (4 + 2 + 1 + 2 + 3 + 3) + (2 + 4) - 8 = 13$